49203be6-68c8-4cae-92ed-2ec0dfa06d

<prev_user>

<request>

You are working on an issue in the 'Azure/typespec-azure' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/typespec-azure/typespec-azure (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-10T06:07:02.391Z</current_datetime>

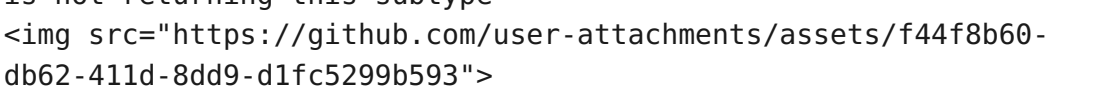
Consider the following problem statement:

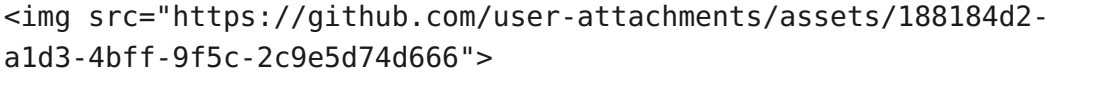
<problem_statement>

This section details on the original issue you should resolve

<issue_title>[Bug]: Discriminated subtype is missing in models from sdkPackages </issue_title>
<issue_description>### Describe the bug

In the following typespec,
```
    @discriminator("kind")
    model Pet {
      kind: string;
      name: string;
      weight?: float32;
    }
    model Cat extends Pet {
      kind: "cat";
      meow: int32;
    }
    model Dog extends Pet {
      kind: "dog";
      bark: string;
```

```
    }
    op read(): { @body body: Cat };
```
The Dog subtype is not referenced by any operation, sdkPackage is not returning this subtype


however, inside the Pet's model definition, it still has Dog's type being referred in the discriminatedSubtypes.


which is confusing to me, if we generate the serializer/deserializer for Pet, we will have to a Dog serializer for it as it's polymorphic base and the discriminator value has a dog in it. but Dog's definition is missing.

I think we could have two possible solutions for it. Depends on how we want to interpret this.
1. delete the Dog subtype from Pet's discriminated subtypes.
2. return the Dog subtype in sdkPackage.models

Reproduction

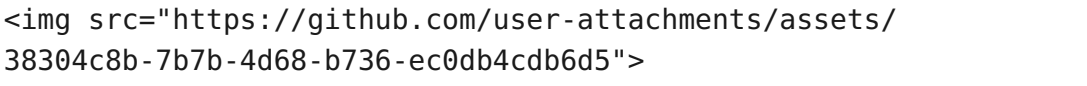
N/A

Checklist

- X] Follow our [Code of Conduct
- X] Check that this issue is about the Azure libraries for typespec. For bug in the typespec language or core libraries file it in the [TypeSpec repo
- [X] Check that there isn't already an issue that request the same bug to avoid creating a duplicate.
- X] The provided reproduction is a [minimal reproducible example of the bug.</issue_description>

<agent_instructions>If only the sub type has usage, we should remove the discriminated info for the base type, thus all sibling types will not be shown to language emitters.</agent_instructions>

Comments on the Issue (you are @copilot in this section)

<comments>
<comment_new><author>@qiaozha</author>
the Dog type we get from from discriminatedSubtypes doesn't have the correct usage flags.

</comment_new>
<comment_new><author>@qiaozha</author>
Summary of conclusion in 2024-09-26 dev office hours meeting:
we should have Dog in models and set the usage the same as in Pet's discriminatedSubtypes. but it should be a low priority thing.</comment_new>
<comment_new><author>@tadelesh</author>
Two solutions:
1. Include all the sub types
2. Remove the discriminated info</comment_new>
</comments>

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

<prev_assistant>

Perfect! Let me now create a comprehensive summary of the work completed:

Summary

I've successfully implemented a fix for the discriminated subtype issue in the Azure TypeSpec repository. Here's what was accomplished:

Problem Statement
When a discriminated base type had subtypes where only some were used in operations, the base model would still reference ALL subtypes in its `discriminatedSubtypes` field, including those with no usage. This caused confusion for language emitters which needed to handle unreferenced types.

Example scenario:

- `Pet` (base with discriminator) has su only the sub type has usage, we should remove the discriminated info for the base type, thus all sibling types will not be shown to language emitters."

I added a `cleanupDiscriminatorForUnusedBase()` function that:
1. Checks all models with `discriminatedSubtypes`
2. Identifies if any subtypes have no usage
3. If so, removes all discriminator metadata from the base model
4. Clears discriminator values from all subtypes in the hierarchy

Key Changes
- **File**: `packages/typespec-client-generator-core/src/types.ts`
  - Added `hasInputOrOutputUsage()` helper function
  - Added `cleanupDiscriminatorForUnusedBase()` function
  - Added `hasBaseModel()` helper with circular reference protection
  - Integrated cleanup into `handleAllTypes()` pipeline

- **File**: `packages/typespec-client-generator-core/test/decorators/usage.test.ts`
  - Added test case "discriminator removed when only subtype is used"

Testing & Validation
✅ All 1176 existing tests pass
✅ New test case validates the fix correctly
✅ CodeQL security scan: 0 alerts
✅ Addressed code review feedback

Impact
Language emitters no longer need to handle discriminated subtypes that aren't included in the SDK package, simplifying code generation and avoiding confusion about missing type definitions.

The implementation is minimal, focused, and follows existing code patterns in the repository.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Azure/typespec-azure' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/

typespec-azure/typespec-azure (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-10T06:30:38.469Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Remove discriminator metadata when subtypes lack usage
</pr_title>

<pr_description>
When a discriminated base model has subtypes where only some are used in operations, the unused subtypes get filtered from `sdkPackage.models` but remain referenced in the base model's `discriminatedSubtypes` map. This creates invalid references to types unavailable to language emitters.

```typespec
@discriminator("kind")
model Pet { kind: string; }
model Cat extends Pet { kind: "cat"; }
model Dog extends Pet { kind: "dog"; }

op read(): { @body body: Cat };  // Only Cat used, Dog has no usage
```

Before: `Pet.discriminatedSubtypes = { cat: Cat, dog: Dog }` but `Dog` not in models list
After: `Pet.discriminatedSubtypes = undefined`

Changes

- **Added `cleanupDiscriminatorForUnusedBase()`** in `types.ts` after usage propagation
  - Detects base models with unused subtypes (no Input/Output usage)
  - Clears `discriminatedSubtypes` and `discriminatorProperty` from base
  - Clears `discriminatorValue` from all subtypes in hierarchy

```
  - Includes circular reference protection in hierarchy
traversal

- **Added helper `hasInputOrOutputUsage()`** for consistent
usage flag checking

- **Test coverage** for Pet/Cat/Dog scenario validates
discriminator removal

Language emitters now see clean model hierarchies without
references to unavailable types.

<issue_title>[Bug]: Discriminated subtype is missing in models
from sdkPackages </issue_title>
&gt;<issue_description>### Describe the bug
&gt;
&gt; In the following typespec,
&gt; ```
&gt;     @discriminator("kind")
&gt;     model Pet {
&gt;       kind: string;
&gt;       name: string;
&gt;       weight?: float32;
&gt;     }
&gt;     model Cat extends Pet {
&gt;       kind: "cat";
&gt;       meow: int32;
&gt;     }
&gt;     model Dog extends Pet {
&gt;       kind: "dog";
&gt;       bark: string;
&gt;     }
&gt;     op read(): { @body body: Cat };
&gt; ```
&gt; The Dog subtype is not referenced by any operation,
sdkPackage is not returning this subtype
&gt;<img src="https://github.com/user-attachments/assets/
f44f8b60-db62-411d-8dd9-d1fc5299b593">
&gt;
&gt; however, inside the Pet's model definition, it still has
Dog's type being referred in the discriminatedSubtypes.
&gt; <img src="https://github.com/user-attachments/assets/
188184d2-a1d3-4bff-9f5c-2c9e5d74d666">
&gt;
&gt; which is confusing to me, if we generate the serializer/
deserializer for Pet, we will have to a Dog serializer for it as
it's polymorphic base and the discriminator value has a dog in
```

it. but Dog's definition is missing.
>
> I think we could have two possible solutions for it. Depends on how we want to interpret this.
> 1. delete the Dog subtype from Pet's discriminated subtypes.
> 2. return the Dog subtype in sdkPackage.models
>
> ### Reproduction
>
> N/A
>
> ### Checklist
>
> - X] Follow our [Code of Conduct
> - X] Check that this issue is about the Azure libraries for typespec. For bug in the typespec language or core libraries file it in the [TypeSpec repo
> - [X] Check that there isn't already an issue that request the same bug to avoid creating a duplicate.
> - X] The provided reproduction is a [minimal reproducible example of the bug.</issue_description>
>
><agent_instructions>If only the sub type has usage, we should remove the discriminated info for the base type, thus all sibling types will not be shown to language emitters.</agent_instructions>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
><comment_new><author>@qiaozha</author>
> the Dog type we get from from discriminatedSubtypes doesn't have the correct usage flags.
> <img src="https://github.com/user-attachments/assets/38304c8b-7b7b-4d68-b736-ec0db4cdb6d5">
> </comment_new>
><comment_new><author>@qiaozha</author>
> ### Summary of conclusion in 2024-09-26 dev office hours meeting:
> we should have Dog in models and set the usage the same as in Pet's discriminatedSubtypes. but it should be a low priority thing.</comment_new>
><comment_new><author>@tadelesh</author>

> Two solutions:
> 1. Include all the sub types
> 2. Remove the discriminated info</comment_new>
></comments>
></pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3776957409</comment_id>
<author>@tadelesh</author>
@copilot I think a better solution is: just remove the model without the usage from the subtype array. The pros is preventing the breaking change.
</comment_new>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>package.json:48</file>
<original_commit>15c852f</original_commit>

<comment_new>
<comment_id>2786038445</comment_id>
<author>@tadelesh</author>
do not change package.json, revert all the changes in this file.
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

The last **5** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address

```
them. Ignore comments that are not actionable or directed at
you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```